

SISTEMA DISTRIBUIDO PARA ANÁLISIS DE SINIESTROS VIALES CON DATOS ABIERTOS DE INEGI

Reporte Técnico del Proyecto Final
Cómputo Paralelo y Distribuido

Tecnologías:	Python · Ray · Ray Cluster · Docker · Pandas · Streamlit
Fuente de datos:	INEGI — ATUS (Accidentes de Tránsito Terrestre)
Fecha:	29 de May de 2026

1. Descripción General

El proyecto implementa una plataforma de análisis distribuido de accidentes viales en México. Utiliza Python como lenguaje base, Ray como framework de computación distribuida, Docker para la orquestación de contenedores y Streamlit para la visualización interactiva de resultados. La aplicación procesa información histórica del INEGI correspondiente a los años 1997-2024 para identificar patrones de riesgo por entidad federativa, municipio, hora, mes, causa y gravedad del accidente.

El sistema responde a la necesidad de instituciones públicas, municipios, aseguradoras y áreas de movilidad urbana que requieren analizar grandes volúmenes de datos de siniestralidad vial. Con más de 28 archivos anuales de datos, el procesamiento secuencial tradicional se vuelve limitado, justificando el uso de procesamiento distribuido.

2. Arquitectura del Sistema

El sistema sigue una arquitectura distribuida de tipo Master-Worker implementada con Ray Cluster sobre contenedores Docker. Se compone de cinco servicios principales:

Componente	Función	Puerto
ray-head	Nodo coordinador del cluster. Administra tareas, workers y objetos compartidos.	6379, 8265
ray-worker-1	Nodo de trabajo. Ejecuta particiones del análisis distribuido.	—
ray-worker-2	Nodo de trabajo. Ejecuta particiones del análisis distribuido.	—
ray-worker-3	Nodo de trabajo. Ejecuta particiones del análisis distribuido.	—
atus-dashboard	Servidor Streamlit. Visualiza los resultados del análisis.	8501

Cada contenedor cuenta con 6 GB de memoria RAM y 2 GB de SWAP, montando el directorio del proyecto como volumen compartido. Esto permite que todos los servicios lean y escriban en la misma carpeta de resultados sin necesidad de transferir archivos entre contenedores.

3. Módulos del Código

3.1 prepare_data.py — Preparación y limpieza de datos

Es el primer módulo en ejecutarse. Su responsabilidad es tomar los archivos CSV crudos del INEGI y producir un dataset limpio y estandarizado en formato Parquet, optimizado para lectura y procesamiento distribuido.

Funciones principales:

- **leer_csv_seguro()**: Lee archivos CSV con manejo automático de encodings (utf-8-sig, utf-8, latin-1) y elimina el caracter BOM que afecta los nombres de columnas en archivos del INEGI.
- **normalizar_columna()**: Convierte nombres de columnas a minúsculas, elimina acentos y caracteres especiales para asegurar compatibilidad entre años.

- **construir_diccionario_entidades() y construir_diccionario_municipios():** Leen los catálogos tc_entidad.csv y tc_municipio.csv del INEGI para traducir códigos numéricos a nombres reales.
- **limpiar_dataset_atus_real():** Mapea columnas, traduce IDs, filtra entidades fuera del rango 1-32, suma columnas de víctimas y genera el índice de gravedad.
- **generar_datos_simulados():** Genera datasets sintéticos con distribuciones estadísticas realistas cuando no hay datos reales disponibles.

El formato de salida Parquet fue elegido sobre CSV porque ocupa considerablemente menos espacio en disco, soporta tipos de datos nativos (fechas, enteros), y se lee mucho más rápido en operaciones de análisis columnar.

3.2 sequential_analysis.py — Análisis con Pandas

Implementa el análisis completo de forma secuencial usando Pandas, la librería estándar de Python para manipulación de datos tabulares. Lee el dataset Parquet completo en memoria y ejecuta todas las agregaciones en un solo proceso.

Análisis que realiza:

Análisis	Método	Resultado
Ranking por entidad	groupby + size	32 estados ordenados por accidentes
Ranking por municipio	groupby + agg	Municipios con heridos, fallecidos e índice
Distribución horaria	groupby + size	Accidentes por cada hora del día (0-23)
Causas frecuentes	groupby + size	Ranking de causas de accidente
Tipos de accidente	groupby + size	Colisión, volcadura, atropellamiento, etc.
Tendencia mensual	groupby + agg	Evolución histórica por año y mes
Zonas de gravedad	groupby + agg	Municipios con mayor índice de gravedad

El índice de gravedad se calcula con la fórmula: **índice = heridos + (fallecidos x 5)**. El factor 5 refleja que un fallecimiento tiene mayor peso social y económico que una lesión.

3.3 ray_analysis.py — Análisis distribuido con Ray

Implementa el mismo análisis pero usando el patrón Map-Reduce distribuido con Ray. Este es el módulo central del proyecto desde el punto de vista de cómputo distribuido.

Flujo de ejecución:

1. **Conexión al cluster:** ray.init(address='auto') conecta al Head Node que coordina los workers.
2. **Particionado:** El DataFrame se divide en 12 particiones de tamaño igual con dividir_dataframe().
3. **Map (fase distribuida):** Cada partición se envía a un worker con analizar_particion.remote(). El decorador @ray.remote indica a Ray que ejecute la función en un nodo remoto.
4. **Ejecución paralela:** Los 3 workers toman las particiones disponibles y las procesan simultáneamente. Con 12 particiones y 3 workers, cada worker procesa aproximadamente 4 particiones.

5. **Reduce (fase de consolidación):** `ray.get()` recolecta los 12 resultados parciales. `reducir_resultados()` los concatena y aplica un segundo `groupby` para obtener los totales finales.

La ventaja de este enfoque es que el procesamiento escala horizontalmente: agregar más workers reduce proporcionalmente el tiempo de análisis. El overhead de coordinación (serialización, transferencia de datos, sincronización) es la principal limitación y explica por qué con datasets pequeños Ray puede ser más lento que Pandas.

3.4 `benchmark.py` — Comparación de rendimiento

Orquesta la ejecución de ambos métodos de forma consecutiva, mide los tiempos con `time.perf_counter()` (el reloj de mayor precisión de Python) y calcula el `speedup`:

```
speedup = tiempo_secuencial / tiempo_ray
```

Un `speedup` mayor a 1 indica que Ray fue más rápido. Los resultados se guardan en `results/benchmark.csv` y `results/benchmark_summary.json` para ser visualizados en el dashboard.

3.5 `app.py` — Dashboard en Streamlit

Aplicación web interactiva que carga los resultados de la carpeta `results/` y los visualiza usando Plotly para gráficas interactivas. Incluye:

- Métricas generales (total accidentes, heridos, fallecidos)
- Comparación de tiempos Pandas vs Ray con gráfica de barras
- Top 15 entidades con más accidentes
- Top 20 municipios con mayor siniestralidad
- Distribución de accidentes por hora del día
- Ranking de causas y tipos de accidente
- Tendencia mensual histórica
- Top 20 zonas con mayor índice de gravedad

4. Fuente de Datos

Campo	Detalle
Institución	INEGI — Instituto Nacional de Estadística y Geografía
Dataset	Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS)
Cobertura temporal	1997 — 2024 (28 archivos anuales)
Cobertura geográfica	Nacional, por entidad federativa y municipio
Variables clave	Fecha, hora, entidad, municipio, causa, tipo, víctimas heridas y fallecidas
Formato de trabajo	Parquet (procesamiento distribuido)
URL	https://www.inegi.org.mx/programas/accidentes/

5. Instrucciones de Ejecución

Requisitos previos:

- Docker Desktop instalado y corriendo
- Git instalado
- Al menos 32 GB de RAM recomendados (6 GB por contenedor x 4 contenedores)

Pasos de ejecución:

1. Clonar el repositorio

```
git clone https://github.com/Tzubay/SDPADSVCDAD-INEGI proyectoinegi cd proyectoinegi
```

2. Crear carpetas necesarias

```
mkdir data\raw data\processed results
```

3. Levantar los contenedores Docker

```
docker compose up --build
```

4. Entrar al contenedor principal

```
docker exec -it ray-head bash
```

5a. Preparar datos simulados

```
python src/prepare_data.py --simulate --rows 200000
```

5b. O preparar datos reales del INEGI

```
python src/prepare_data.py
```

6. Correr el benchmark

```
python src/benchmark.py
```

7. Ver el dashboard

Abrir navegador en: <http://localhost:8501>

8. Apagar los contenedores

```
docker compose down
```

6. Indicadores del Sistema

Indicador	Descripción
Total de accidentes	Número total de registros analizados en el dataset.
Accidentes con heridos	Registros donde al menos una persona resultó lesionada.
Accidentes con fallecidos	Registros donde hubo al menos una víctima mortal.
Accidentes por hora	Distribución de siniestros por hora del día (0 a 23).
Accidentes por municipio	Ranking municipal de siniestralidad con heridos y fallecidos.
Accidentes por causa	Causas más frecuentes: conductor, falla vehicular, camino, etc.
Accidentes por tipo	Colisión, volcadura, atropellamiento, salida del camino u otro.
Índice de gravedad	heridos + (fallecidos x 5) por zona geográfica.
Tendencia anual	Evolución histórica mensual de la siniestralidad desde 1997.
Speedup Ray vs Pandas	$\text{tiempo_secuencial} / \text{tiempo_ray}$. Mide la ganancia del procesamiento distribuido.

7. Conclusiones

El proyecto demuestra de forma práctica las ventajas y limitaciones del procesamiento distribuido aplicado a datos reales de interés público. Con un dataset de más de 6 millones de registros históricos de accidentes viales, Ray Cluster ofrece una reducción significativa en los tiempos de análisis respecto a Pandas secuencial.

La arquitectura implementada es escalable horizontalmente: agregar nodos worker al cluster permite procesar volúmenes de datos aún mayores sin modificar el código. El patrón Map-Reduce aplicado garantiza que los resultados sean idénticos a los del análisis secuencial, solo que obtenidos en menos tiempo.

Desde la perspectiva del análisis de datos, el sistema permite identificar patrones relevantes para la política pública: las entidades con mayor siniestralidad, los horarios de mayor riesgo, las causas más frecuentes y las zonas con mayor índice de gravedad. Esta información tiene aplicación directa en áreas de movilidad urbana, seguridad vial y aseguradoras.

Como trabajo futuro se podría extender el sistema para procesar datos en tiempo real usando Ray Serve, integrar modelos de predicción de accidentes, y desplegar el cluster en infraestructura cloud para mayor capacidad de cómputo.

